

Chapter 5B: CSP Search (Part 1)

AMCS2104 Fundamentals of Artificial Intelligence

Content

- AC-3 Algorithm
- Path Consistency
- Global Constraint
- Resource Constraint
- Bound Constraint
- Sudoku Problem
- Backtracking Search for CSPs
- Variable Ordering
- Value Ordering

Algorithm for Arc Consistency: AC-3

- **AC-3 algorithm** is the most popular algorithm for enforcing arc consistency.
- It maintains a queue of arcs to consider to make every variable arc-consistent until no more arcs are in the queue.
 - Initially, the queue contains all the arcs in the CSP.
 - Each binary constraint becomes two arcs, one in each direction.
 - AC-3 then pops off an arbitrary arc (X_i, X_j) from the queue and makes X_i arc-consistent with respect to X_j .
 - Impact on domain values:
 - If D_i remains unchanged, AC-3 moves on to the next arc.

Algorithm for Arc Consistency: AC-3

- If D_i is reduced, all arcs (X_k, X_i) where X_k is a neighbor of X_i is added to the queue as changes in D_i might enable further reductions in the domains of D_k , even if we have previously considered X_k .
- If D_i is revised down to nothing, it means the whole CSP has no consistent solution, and AC-3 can immediately return failure.
- At the end of the inference:
 - The resulting CSP is still equivalent to the original CSP.
 - Both have the same solutions.
 - However, an arc-consistent CSP will have smaller domains, which result in a more efficient search.

function AC-3(csp) **returns** false if an inconsistency is found and true otherwise
 $queue \leftarrow$ a queue of arcs, initially all the arcs in csp

while $queue$ is not empty **do**
 $(X_i, X_j) \leftarrow \text{POP}(queue)$
 if REVISE(csp, X_i, X_j) **then**
 if size of $D_i = 0$ **then return** false
 for each X_k **in** $X_i.\text{NEIGHBORS} - \{X_j\}$ **do**
 add (X_k, X_i) to $queue$
return true

function REVISE(csp, X_i, X_j) **returns** true iff we revise the domain of X_i
 $revised \leftarrow$ false
for each x **in** D_i **do**
 if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j **then**
 delete x from D_i
 $revised \leftarrow$ true
return revised

Figure 5.3 The arc-consistency algorithm AC-3. After applying AC-3, either every arc is arc-consistent, or some variable has an empty domain, indicating that the CSP cannot be solved. The name “AC-3” was used by the algorithm’s inventor (Mackworth, 1977) because it was the third version developed in the paper.

Simulation of AC-3

- Consider the following CSP where:
 - Variables and domains:
 - $X: \{0, 1, \dots, 9, 10\}$
 - $Y: \{0, 1, \dots, 9, 10\}$
 - Constraint: $\langle (X, Y), Y = X^2 \rangle$

- Get ready all the arcs from the constraints:
 - $Y = X^2$
 - $X^2 = Y$ or $X = \sqrt{Y}$

Simulation of AC-3

□ At the beginning:

□ $X: \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

□ $Y: \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

□ Queue: $\{X = \sqrt{Y}, Y = X^2\}$

□ Step 1:

□ $X: \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

□ $Y: \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

□ Arc: $\{X = \sqrt{Y}\}$

□ Queue: $\{Y = X^2\}$

Simulation of AC-3

□ At the end of Step 1:

□ $X: \{0, 1, 2, 3\}$

□ $Y: \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

□ Arc: $\{\}$

□ Queue: $\{Y = X^2\}$

X is now arc-consistent
with respect to Y



□ Step 2:

□ $X: \{0, 1, 2, 3\}$

□ $Y: \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

□ Arc: $\{Y = X^2\}$

□ Queue: $\{\}$

D_X is reduced, so all arcs (X_k, X)
where X_k is a neighbor of X is
added to the queue as changes
in D_X might enable further
reductions in the domains of D_{X_k}



Simulation of AC-3

□ At the end of Step 2:

□ $X: \{0, 1, 2, 3\}$

□ $Y: \{0, 1, 4, 9\}$

□ Arc: $\{\}$

□ Queue: $\{\}$ ← $X = \sqrt{Y}$

Y is now arc-consistent
with respect to X

□ At the end of Step 2 (cont.):

□ $X: \{0, 1, 2, 3\}$

□ $Y: \{0, 1, 4, 9\}$

□ Arc: $\{\}$

□ Queue: $\{X = \sqrt{Y}\}$

D_Y is reduced, so all arcs (X_k, Y)
where X_k is a neighbor of Y is
added to the queue as changes
in D_Y might enable further
reductions in the domains of D_{X_k}

Simulation of AC-3

□ Step 3:

□ $X: \{0, 1, 2, 3\}$

□ $Y: \{0, 1, 4, 9\}$

□ Arc: $\{X = \sqrt{Y}\}$

□ Queue: $\{\}$

□ At the end of Step 3:

□ $X: \{0, 1, 2, 3\}$

□ $Y: \{0, 1, 4, 9\}$

□ Arc: $\{\}$

□ Queue: $\{\}$

Complexity of AC-3

- Complexity of AC-3
 - Assume a CSP with n variables, each with domain size at most d , and with c binary constraints (arcs).
 - Each arc (X_k, X_i) can be inserted in the queue only d times because X_i has at most d values to delete.
 - Checking consistency of an arc can be done in $O(d^2)$ time.
 - The total worst-case time is $O(cd^3)$.

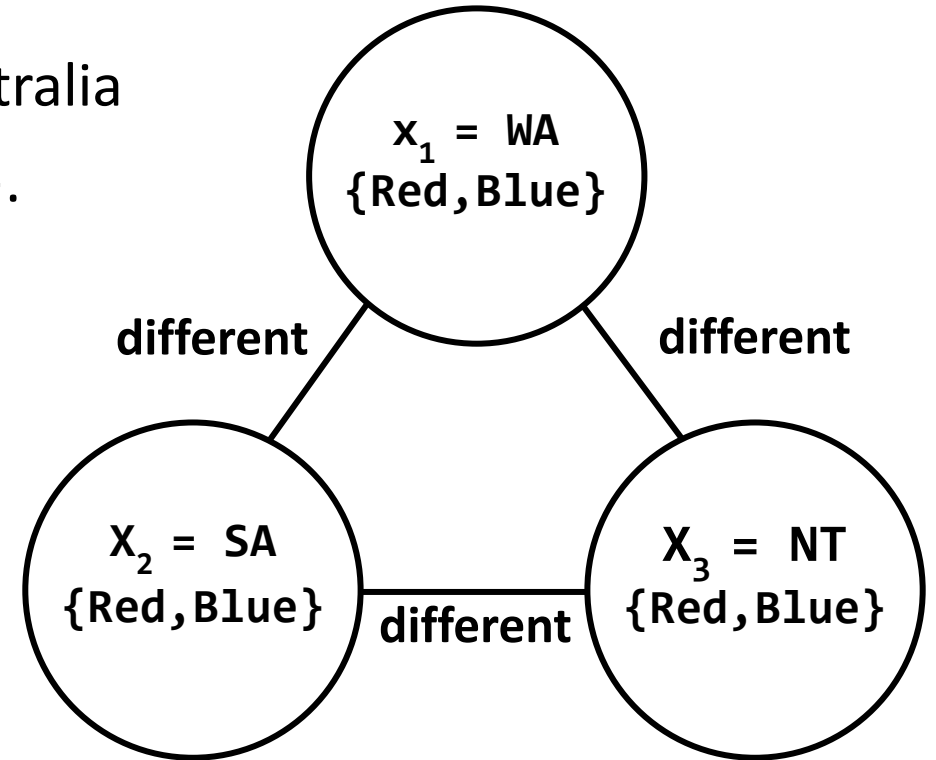
Arc Consistency: Limitation

- Although arc consistency can reduce the domains of variables...
 - Sometimes it finds a solution (every domain is reduced to size 1).
 - Sometimes it finds that the CSP cannot be solved (some domains are reduced to size 0).
 - Sometimes it fails to make enough inferences for some graphs and hence fails to discover the fact that solution doesn't exist.

Arc Consistency: Limitation

□ Example

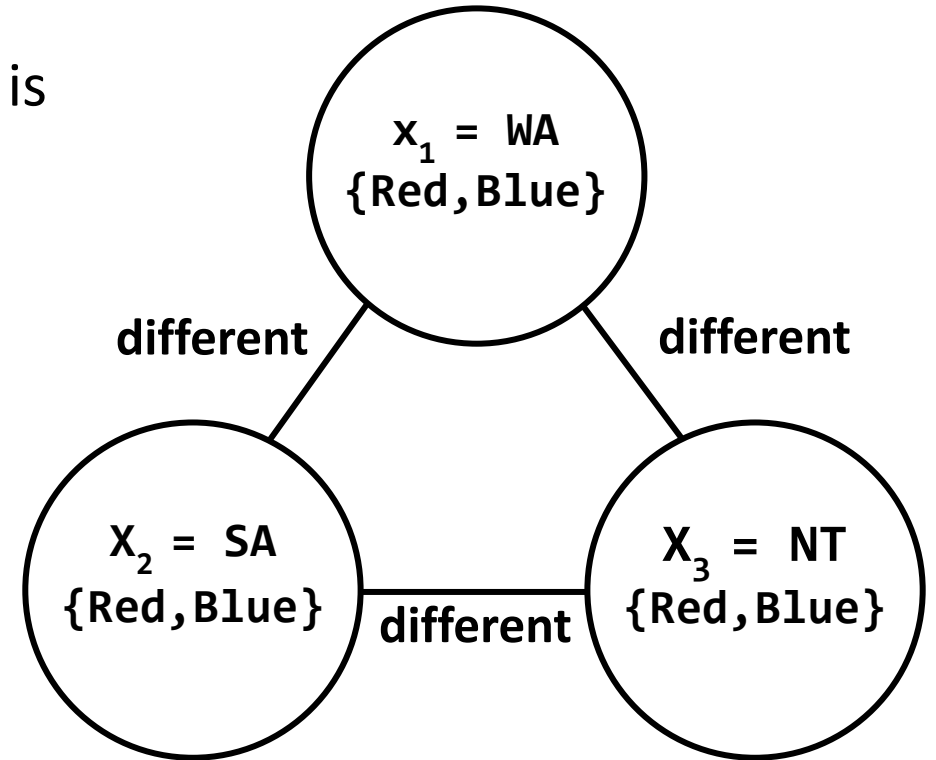
- Consider the map-coloring problem on Australia with only two colors allowed, {red, blue}.
- This CSP is arc-consistent because every variable is already arc consistent.
- But obviously, there is no solution.



Arc Consistency: Limitation

□ Example

- One of the relationships between x_1 and x_2 is that $\{WA = Red\}$ and $\{SA = Blue\}$.
- For this pair of assignment, x_3 has no arc-consistent value corresponding to it.
- Yet, it cannot be concluded with the use of arc-consistency.
- Path consistency can be used to solve this problem.



Path Consistency

- **Path consistency** is a stronger notion of consistency.
- It tightens the binary constraints by using implicit constraints that are inferred by looking at triples of variables.
- A two-variable set $\{X_i, X_j\}$ is path-consistent **with respect to a third variable X_m** if, for every assignment $\{X_i = a, X_j = b\}$ consistent with the constraints on $\{X_i, X_j\}$, there is an assignment to X_m that satisfies the constraints on $\{X_i, X_m\}$ and $\{X_m, X_j\}$
- The name refers to the overall consistency of the path from X_i to X_j with X_m in the middle.

Path Consistency

- Example: Map Coloring Problem
 - Suppose the set $\{WA, SA\}$ is arc consistent.
 - So, the set $\{WA, SA\}$ will be made path consistent with respect to NT.
 - Enumerating the consistent assignments to the set results in:
 $\{WA = \text{Red}, SA = \text{Blue}\}$ and $\{WA = \text{Blue}, SA = \text{Red}\}$
 - Provided these two assignments, NT can be neither $\{\text{Red}\}$ nor $\{\text{Blue}\}$.
 - For consistency purpose, these two assignments are eliminated.
 - As a result, there is no valid assignment for $\{WA, SA\}$.
 - Thus, there is no solution to this problem.

Global Constraint

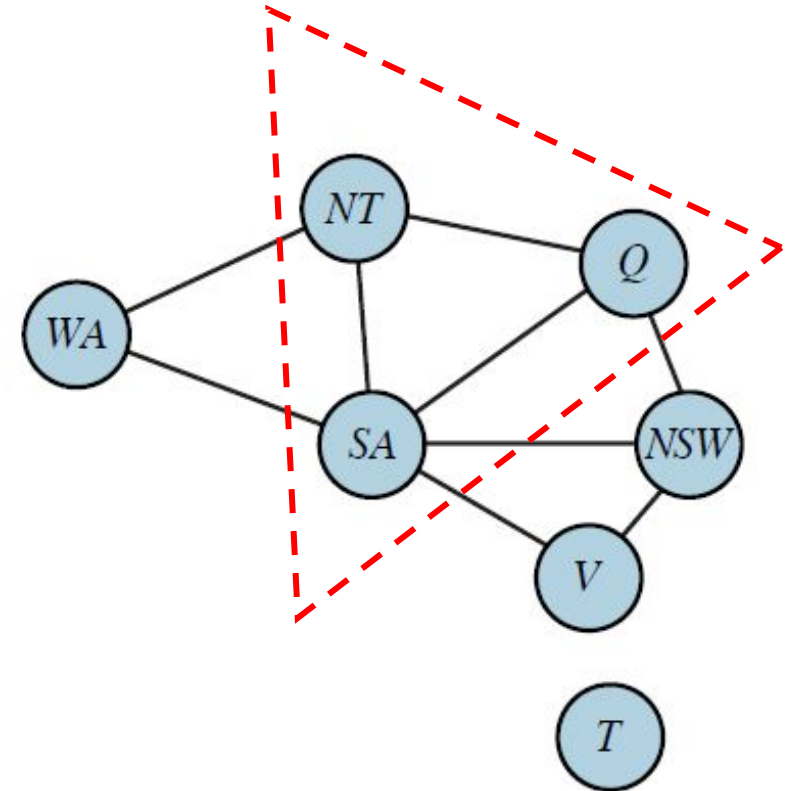
- A **global constraint** is one that involves an **arbitrary number of variables**.
- It is also referred to as **higher-order constraint**.
- An example of the global constraint is the `Alldiff` constraint.
- One simple form of inconsistency detection for `Alldiff` works as follows:
 - If **m** variables are involved in the constraint,
 - If they have **n** possible distinct values altogether, and
 - If **$m > n$** ,
 - Then the constraint cannot be satisfied.

Global Constraint Algorithm

- First, remove any variable in the constraint that has a singleton domain.
- Delete that variable's value from the domains of the remaining variables.
- Repeat as long as there are singleton variables.
- If at any point an empty domain is produced or there are more variables than domain values left, then an inconsistency has been detected.

Global Constraint

- Example: Map Coloring Problem
 - This method can detect inconsistency in the assignment:
 $\{WA = \text{red}, NSW = \text{red}\}$
 - After applying AC-3 with the partial assignment, the domain of each variable is reduced to:
 $\{\text{green}, \text{blue}\}$
 - Now, there are only three variables and two colors, so the Alldiff constraint is violated.



Resource Constraint

- **Resource constraint** is another important higher-order constraint, sometimes called the Atmost constraint.
- Example: Scheduling Problem
 - $X: \{T_1, T_2, T_3, T_4\}$ denote the # of staff assigned to each of four tasks.
 - C: No more than 10 staff are assigned in total is written as:
$$\text{Atmost}(10, T_1, T_2, T_3, T_4)$$
 - Inconsistency can be detected by checking the sum of the minimum values of the current domains, for instance if:
 - $D: \{3, 4, 5, 6\}$, then the Atmost constraint cannot be satisfied.

Resource Constraint

- **Question:** Why the Atmost constraint cannot be satisfied?
- Consistency can be enforced by deleting the maximum value of any domain if it is not consistent with the minimum values of the other domains.
- Example
 - If each variable in the example has the domain {2, 3, 4, 5, 6}, the values 5 and 6 can be deleted from each domain.

Bound Constraint

- For problems with a large set of integer values, domains are represented by upper and lower bounds and are managed by **bounds propagation**.
 - E.g., logistical problems involving moving thousands of people in hundreds of vehicles.
- A CSP is bounds-consistent if for every variable X , and for both the lower-bound and upper-bound values of X , there exists some value of Y that satisfies the constraint between X and Y for every variable Y .
- This kind of bounds propagation is widely used in practical constraint problems.

Bound Constraint

□ Example: Airline-scheduling problem

□ Suppose there are two flights, F_1 and F_2 .

□ Each plane has the capacities 165 and 385, respectively.

□ The initial domains for the numbers of passengers on flights F1 and F2 are:

$$D_1 = [0, 165] \text{ and } D_2 = [0, 385]$$

□ Now suppose we have the additional constraint that the two flights together must carry 420 people:

$$F_1 + F_2 = 420$$

□ Propagating bounds constraints, we reduce the domains to:

$$D_1 = [35, 165] \text{ and } D_2 = [255, 385]$$

Sudoku

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

Sudoku

- The popular Sudoku puzzle has introduced millions of people to CSPs without having them realizing it.

- **Sudoku puzzle**

- It consists of 81 squares, some of which are initially filled with digits from 1 to 9.
 - The solution is to fill in all the remaining squares such that no digit appears twice in any row, column, or 3×3 box.
 - A row, column, or box is called a unit.

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

Sudoku

- A Sudoku can be formulated as CSP:

- Variables

- There are 81 variables, one for each square.

- The variables are named with row and number such as.

- A1 and A9 (in the top row).

- I1 and I9 (in the bottom row).

- Domains

- The empty squares have the domain {1, 2, 3, 4, 5, 6, 7, 8, 9}.

- The prefilled squares have a domain consisting of a single value.

Sudoku

□ Constraints

- There are 27 different Alldiff constraints, one for each row, column, and box of 9 squares:

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

Alldiff(A1, A2, A3, A4, A5, A6, A7, A8, A9)

Alldiff(B1, B2, B3, B4, B5, B6, B7, B8, B9)

...

Alldiff(A1, B1, C1, D1, E1, F1, G1, H1, I1)

Alldiff(A2, B2, C2, D2, E2, F2, G2, H2, I2)

...

Alldiff(A1, A2, A3, B1, B2, B3, C1, C2, C3)

Alldiff(A4, A5, A6, B4, B5, B6, C4, C5, C6)

Sudoku

- Consider variable E6:
 - The box constraints allow the removal of {1, 2, 7, 8} from its domain:
 - $D_{E6} = \{3, 4, 5, 6, 9\}$.
 - The column constraints allow the removal of {2, 3, 5, 6, 8, 9}.
 - As a result, E6 only has a domain of {4}.

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

Sudoku

- Now consider variable I6:
 - The column constraints allow the removal of {2, 3, 4, 5, 6, 8, 9}.
 - The box constraints allow the removal of {1, 2, 3, 6, 9}.
 - As a result, I6 is left with a single domain value of {7} only.
- Now consider variable A6:
 - Since there are 8 known values in column 6, so the answer must be {1}.

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7					4			8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1	7	3		

Sudoku

- Inference continues along these lines, and eventually, the entire puzzle can be solved where all the variables have their domains reduced to a single value.
- The AC-3 algorithm can be applied directly to solve the Sudoku problem if Alldiff constraints have been expanded into binary constraints (such as $A1 \neq A2$).

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

Search for CSPs

- Some CSPs can be solved by inference over constraints, others can't.
- Thus, algorithms have to be applied to search for a solution.
- There are two types of search algorithms:
 - Backtracking search algorithms
 - Work on partial assignments
 - Local search algorithms
 - Work on complete assignments

Depth-Limited Search for CSPs

- Depth-limited search
 - State: A partial assignment.
 - Action: Extension of an assignment.
- Issue
 - A CSP with n variables of domain size d has a **branching factor** of:
 - At the top level: **nd** (any d values can be assigned to any **n**)
 - At the next level: **$(n - 1)d$** , and so on for n levels
 - At the 2nd last bottom level: **$2d$**
 - At the bottom level: **d**
 - Total number of tree nodes: **$n! \times d^n$**

Search for CSPs

- A problem is **commutative** if the order of application of any given set of actions has no effect on the outcome.
- Example
 - Commutative: $3 + 5 = 5 + 3 = 8$
 - Non-commutative: $10 \div 5 \neq 5 \div 10$
- For classical search, where the solution is a sequence of action, the problem is non-commutative.
- However, all CSPs are **commutative** because when assigning values to variables, the same partial assignment is reached regardless of order.
- Thus, we need to consider only a **single variable at each node** in the search tree.

Search for CSPs

- Example: CSPs are commutative
 - The first sequence of assignment
 - {WA = Red} □
 - {WA = Red, NT = Green} □
 - {WA = Red, NT = Green, SA = Blue}
 - The second sequence of assignment
 - {NT = Green} □
 - {NT = Green, WA = Red} □
 - {NT = Green, WA = Red, SA = Blue}
 - The same partial assignment is reached at the end.

Search for CSPs

- Example: Map Coloring Problem

- At the root node, we might make a choice between

- $\{SA = \text{Red}, SA = \text{Green}, \text{ and } SA = \text{Blue}\}$

- Avoid making decision between

- $\{SA = \text{Red and } WA = \text{Blue}\}$

- With this restriction, the number of leaves might be reduced to d^n .

Search for CSPs

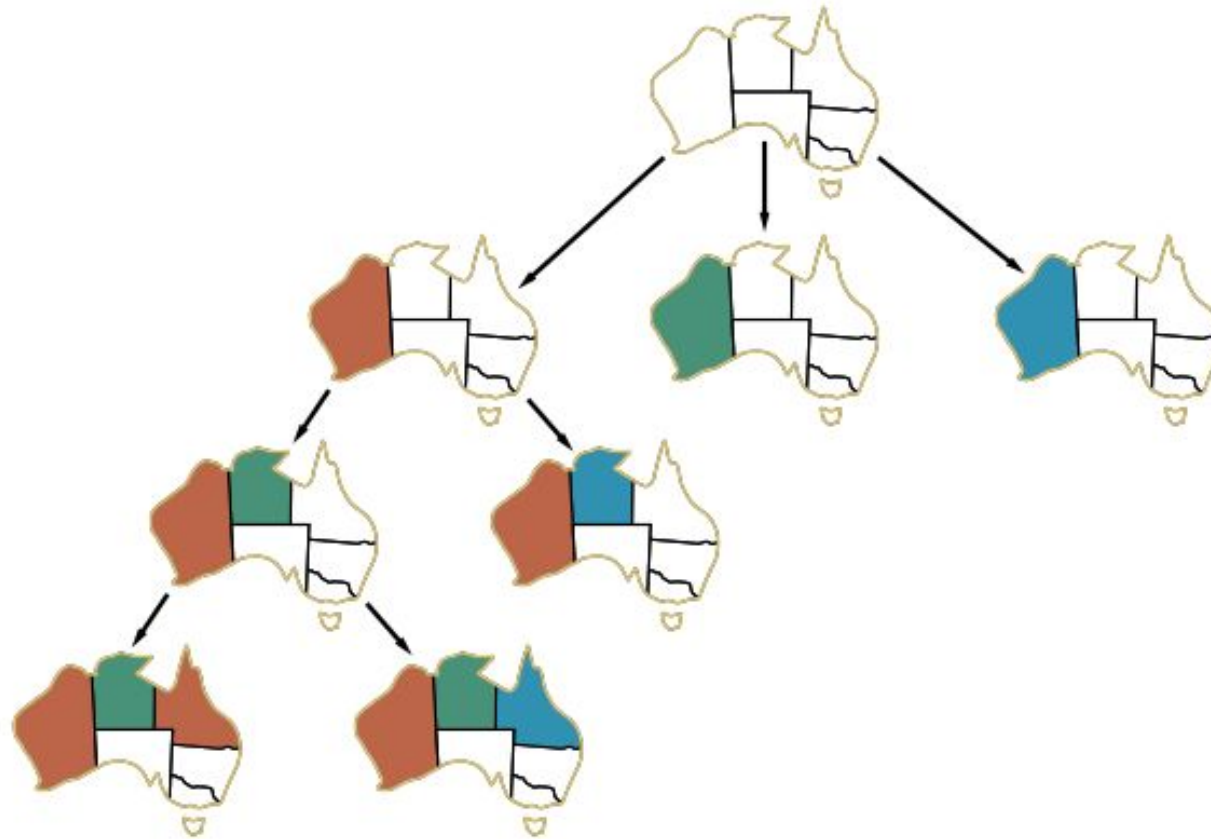


Figure 5.6 Part of the search tree for the map-coloring problem in Figure 5.1.

Backtracking Search Algorithm

- A **backtracking search** is a variant of **depth-first search** that chooses values for one variable at a time and **backtracks when a variable has no legal values left to assign**:
 - It repeatedly chooses an unassigned variable.
 - Then, it tries all values in the domain of that variable in turn, trying to extend each one into a solution via a recursive call.
 - If the call succeeds, the solution is returned.
 - If it fails, the assignment is restored to the previous state, and we try the next value.
 - If no value works, then we return failure.

Backtracking Search Algorithm

- **Backtracking search** keeps only a single representation of a state and alters that representation.
- Unlike uninformed search, it doesn't rely on **domain-specific heuristic**.

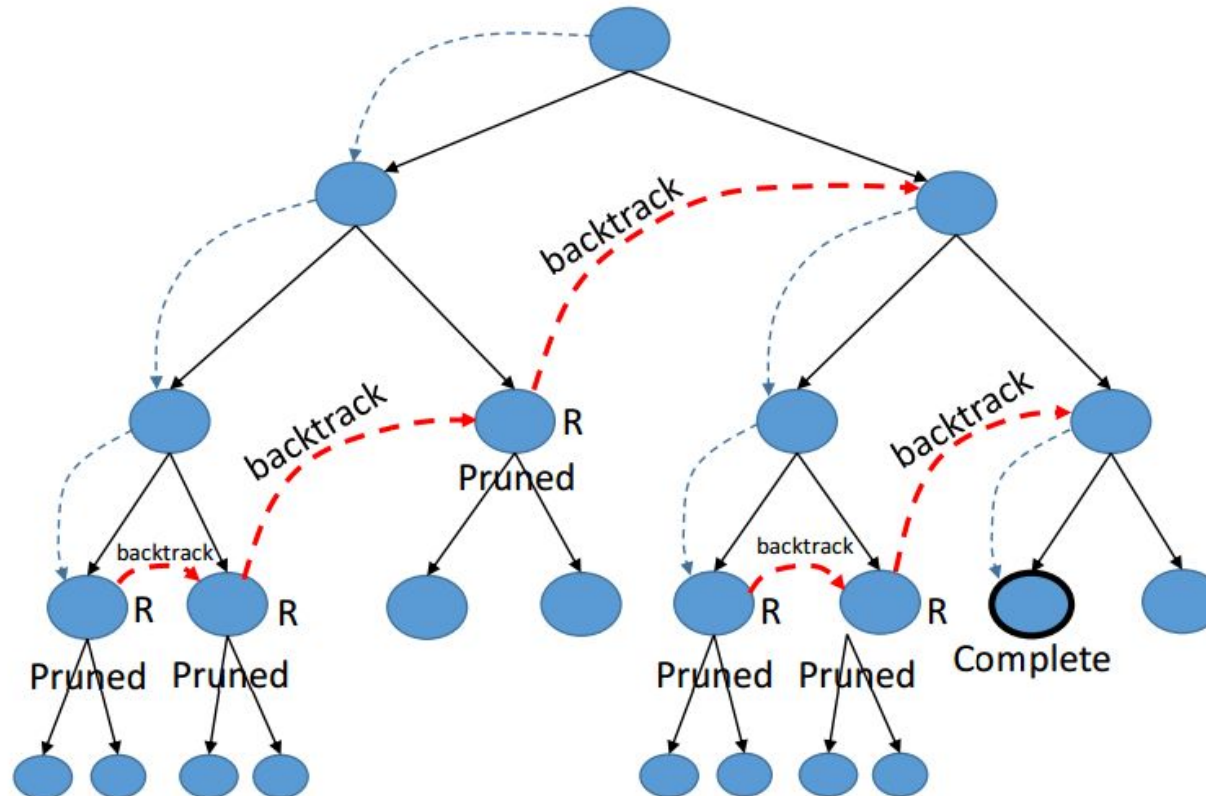
```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
  return BACKTRACK(csp, { })

function BACKTRACK(csp, assignment) returns a solution or failure
  if assignment is complete then return assignment
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(csp, assignment)
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences  $\leftarrow$  INFERENCE(csp, var, assignment)
      if inferences  $\neq$  failure then
        add inferences to csp
        result  $\leftarrow$  BACKTRACK(csp, assignment)
        if result  $\neq$  failure then return result
        remove inferences from csp
      remove {var = value} from assignment
  return failure
```

Figure 5.5 A simple backtracking algorithm for constraint satisfaction problems. The algorithm is modeled on the recursive depth-first search of Chapter 3. The functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES implement the general-purpose heuristics discussed in Section 5.3.1. The INFERENCE function can optionally impose arc-, path-, or k -consistency, as desired. If a value choice leads to failure (noticed either by INFERENCE or by BACKTRACK), then value assignments (including those made by INFERENCE) are retracted and a new value is tried.

Backtracking Search vs. Depth-First Search

- Backtracking is usually implemented as a combination of depth-first search and pruning.



Backtracking Search: Example #1

- Simulate how backtracking search works on the following CSP:
 - Variables: $\{X, Y, Z\}$
 - Domain for each variable: $\{1, 2, 3\}$
 - Constraints:
 - $X = Y - 1$
 - $Y = Z$

Backtracking Search: Example #1

□ At the beginning:

□ Variables

□ $X: \{1, 2, 3\}$

□ $Y: \{1, 2, 3\}$

□ $Z: \{1, 2, 3\}$

□ Constraints

□ $X = Y - 1$

□ $Y = Z$

Pruned...

□ $X = 1, Y = 1, Z = 1$

□ $X = 1, Y = 1, Z = 2$

□ $X = 1, Y = 1, Z = 3$

□ $X = 1$

□ $X = 1, (Y) = 1$ ✗

□ $X = 1, Y = 2$

□ $X = 1, Y = 2, (Z) = 1$ ✗

□ $X = 1, Y = 2, Z = 2$ ✓

Backtracking Search: Example #2

- Simulate how backtracking search works on the following CSP:
 - Variables: $\{X, Y, Z\}$
 - Domain for each variable: $\{1, 2, 3\}$
 - Constraints:
 - $X > Y$
 - $Y \neq Z$
 - $Z > X$

Backtracking Search: Example #2

□ At the beginning:

□ Variables

□ $X: \{1, 2, 3\}$

□ $Y: \{1, 2, 3\}$

□ $Z: \{1, 2, 3\}$

□ Constraints

□ $X > Y$

□ $Y \neq Z$

□ $Z < X$

□ $X = 1$

□ $X = 1, Y = 1$ ✗

□ $X = 1, Y = 2$ ✗

□ $X = 1, Y = 3$ ✗

□ $X = 2$

□ $X = 2, Y = 1$

□ $X = 2, Y = 1, Z = 1$ ✗

□ $X = 2, Y = 1, Z = 2$ ✗

□ $X = 2, Y = 1, Z = 3$ ✗

□ ...

Backtracking Search: Example #2

- At the beginning:

- Variables

- $X: \{1, 2, 3\}$

- $Y: \{1, 2, 3\}$

- $Z: \{1, 2, 3\}$

- Constraints

- $X > Y$

- $Y \neq Z$

- $Z < X$

- ...

- $X = 3$

- $X = 3, Y = 2$

- $X = 3, Y = 2, Z = 1$ ✓

Backtracking Search Algorithm

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return BACKTRACK( $\{\}$ , csp)

function BACKTRACK(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences  $\leftarrow$  INFERENCE(csp, var, value)
      if inferences  $\neq$  failure then
        add inferences to assignment
        result  $\leftarrow$  BACKTRACK(assignment, csp)
        if result  $\neq$  failure then
          return result
      remove {var = value} and inferences from assignment
  return failure
```

Backtracking Search Algorithm

- Backtracking algorithm can be improved using domain-independent heuristics :
 - Which variable should be assigned next?
 - `SELECT-UNASSIGNED-VARIABLE()`
 - What order should its values be tried?
 - `ORDER-DOMAIN-VALUES()`
 - What inferences should be performed at each step in the search?
 - `INFERENCE()`
 - Can we BACKTRACK more than one step when appropriate?
 - `BACKTRACK()`
 - How to avoid repeat the same constraint violation during an assignment?

Variable Ordering

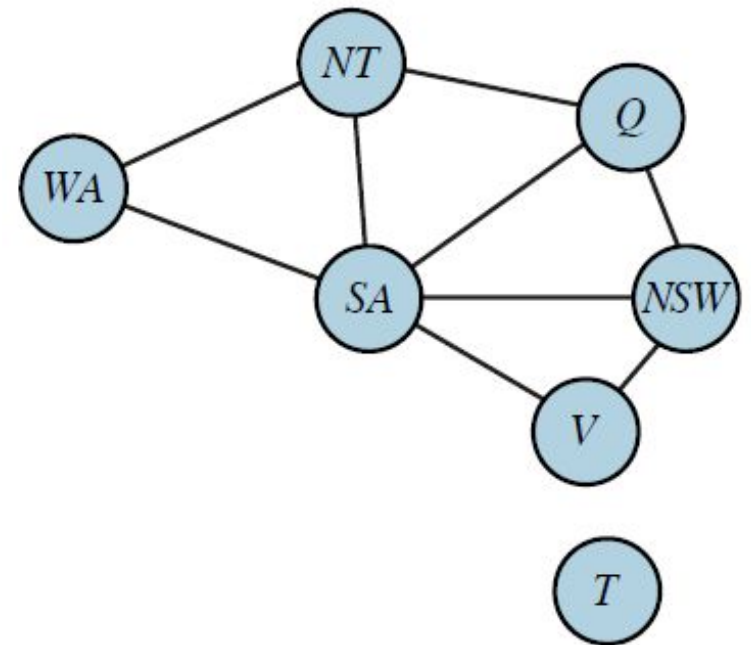
- The simplest strategy is to choose the next unassigned variable in **static/random ordering**:

$$\{X_1, X_2, \dots\}$$

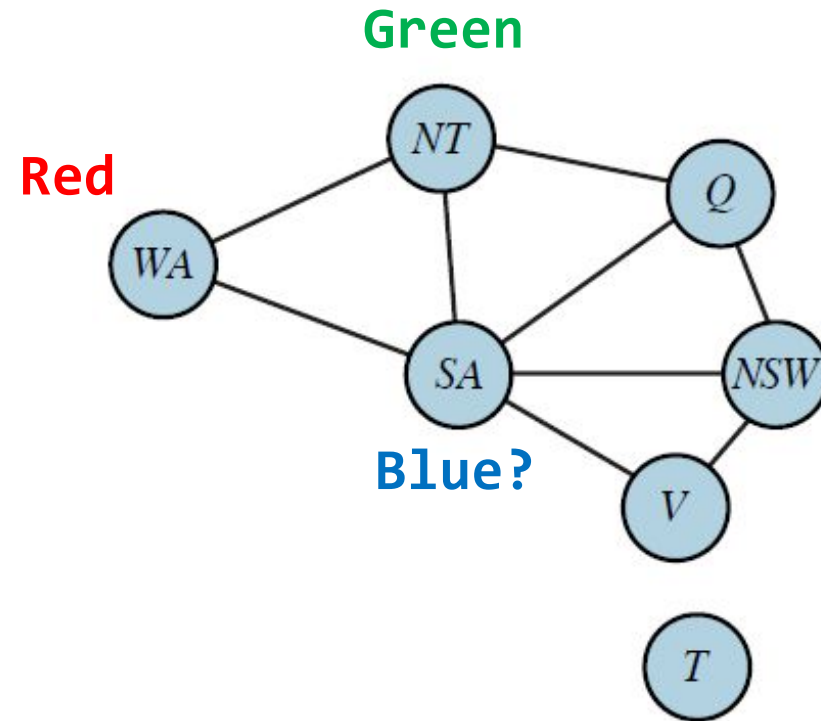
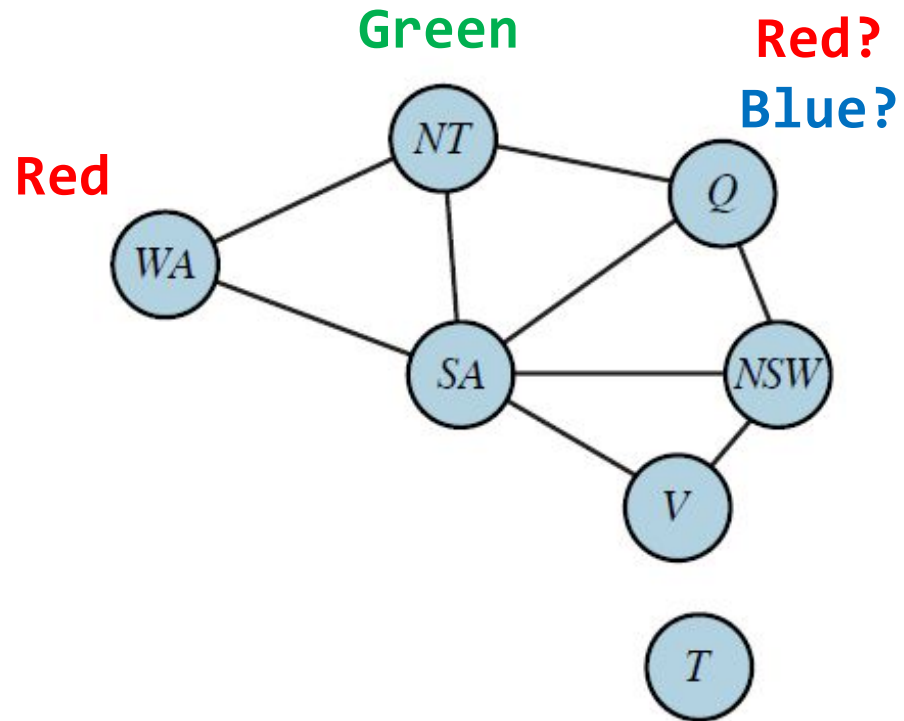
- Example

- The assignments $\{WA = \text{Red}, NT = \text{Green}\}$ come first.
- The assignment for Q comes next.

- **Question:** What will happen if Q or other variables are assigned first?



Variable Ordering



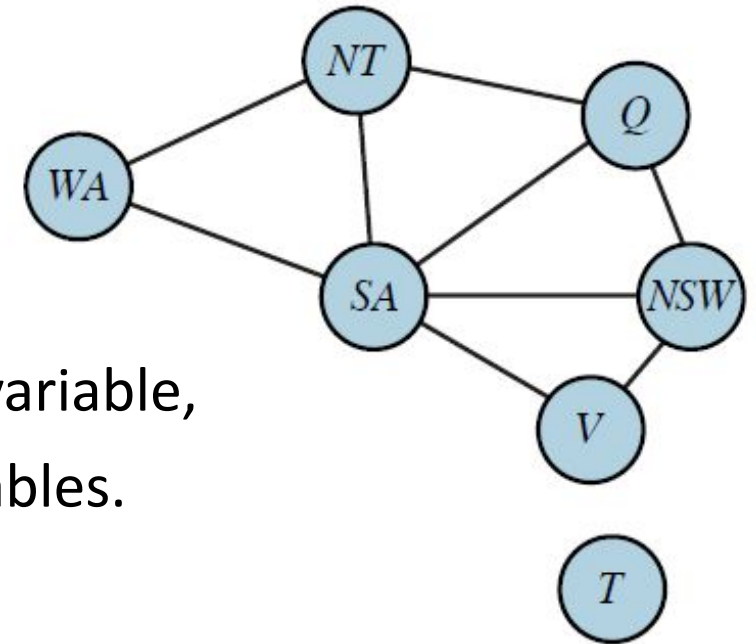
□ The result of static and random ordering is not optimal.

Variable Ordering

- The **minimum-remaining-values (MRV) heuristic** chooses variable with the fewest "legal" values.
- It tends to pick a variable that is most likely to cause a failure.
- Thus, it is also called the "**most constrained variable**" or "**fail-first**" heuristic.
- Example
 - The MRV heuristic will select X if it has no legal values left.
 - Failure can be detected immediately.
- Overall, the MRV heuristic outperforms a random or static ordering, sometimes by a factor of 1,000 or more.

Variable Ordering

- ❑ **Question:** Why MRV heuristics is only applied after the assignment of { WA = red and NT = green }, but not applied at the beginning?
- ❑ The MRV heuristic doesn't work in choosing the first variable, because initially all domain values are legal to all variables.
- ❑ **Question:** Which variable should be chosen first?



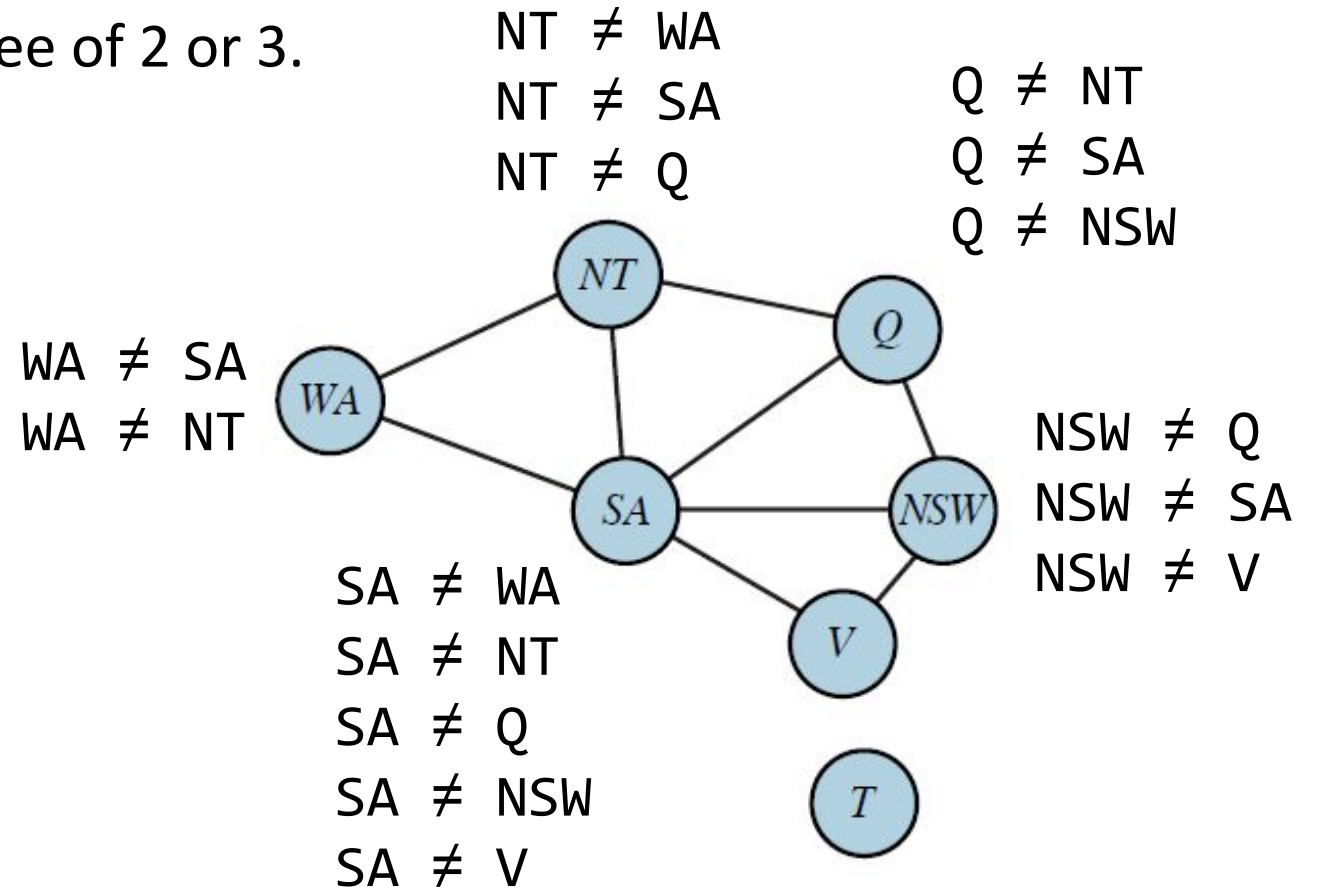
Variable Ordering

□ Degree heuristic

- Attempts to **reduce the branching factor on future choices**.
- Selects variable involved in **the largest number of constraints** on other unassigned variables.
- Example
 - SA is the variable with highest degree, 5.
 - The other variables have a degree of 2 or 3.
 - T has the lowest degree, 0.
- Normally, once the most constraint variable is chosen first, the problem can be solved without any false steps.

Variable Ordering

- SA is the variable with highest degree, 5.
- The other variables have a degree of 2 or 3.
- T has the lowest degree, 0.



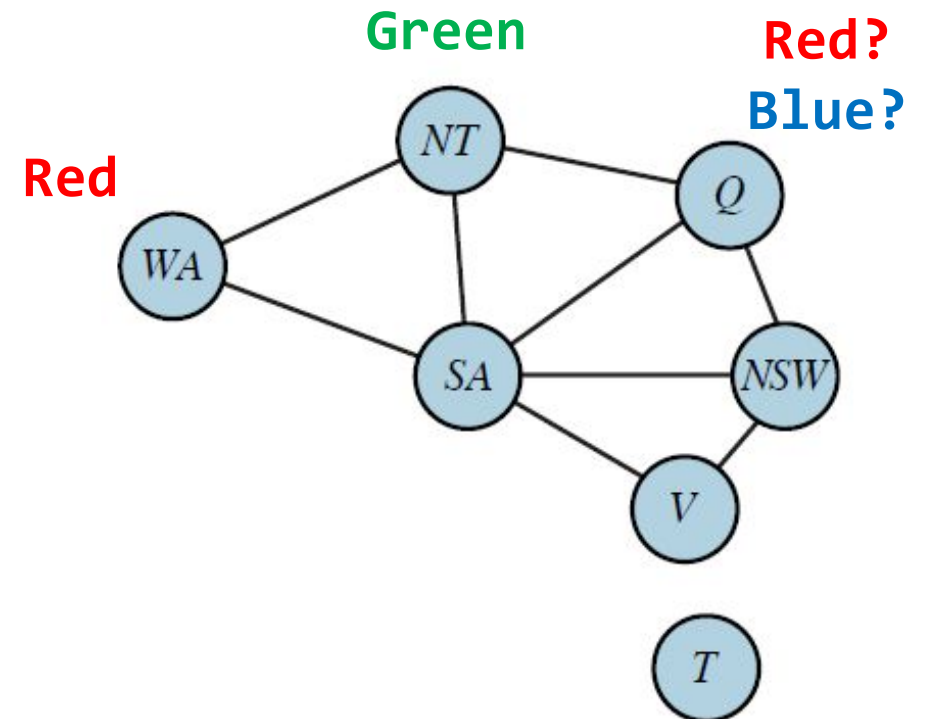
Variable Ordering

- MRV heuristic
 - A powerful guide
- Degree heuristic
 - A tie-breaker

Value Ordering

- For value ordering, the **least-constraining-value heuristic** prefers any value that rules out the fewest choices for the neighboring variables in the constraint graph.

- Example
 - Consider the given partial assignment, what is the best value for Q ?
 - $\{Q = \text{Blue}\}$ is a bad choice as it eliminates the last legal value left for SA .



Value Ordering

□ Example #1

□ {WA = Red, NT = Green, Q = Blue}

□ SA = {}, NSW = {Red, Green}, V = {Red, Green, Blue}, T = {All}

□ Example 2

□ {WA = Red, NT = Green, Q = Red}

□ SA = {Blue}, NSW = {Green}, V = {Red}, T = {All}

□ Thus, the least-constraining-value heuristic prefers {Red} to {Blue}.

□ The heuristic tries to **leave the maximum flexibility for subsequent variable assignments.**

Variable Ordering vs. Value Ordering

- The **minimum-remaining-values (MRV) heuristic** chooses variable with the fewest "legal" values.
- The **least-constraining-value heuristic** prefers any value that rules out the fewest choices for the neighboring variables in the constraint graph.
- **Question:** Why should variable selection be fail-first, but value selection be fail-last?

Variable Ordering vs. Value Ordering

- Variable ordering: Fail-first
 - Every variable has to be assigned eventually.
 - By choosing the ones that are likely to fail first, we will on average have fewer successful assignments to backtrack over.
 - Thus, it improves search efficiency.
- Value ordering: Fail-last
 - The ultimate goal of solving a CSP is to find out one solution.
 - Starting from fail-first value beats the purpose.
 - Therefore, it makes sense to look for the most likely values first.

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 5: Constraint Satisfaction Problems (page 170 to page 178)

The End